

RPCW2025/semana2

9 a 13 de Fevereiro de 2026

- Tutorial sobre o Protégé: [o Cinema](#);
 - Exercício prático: carregar alguns indivíduos na ontologia do cinema;
 - Exercício prático: especificação de uma ontologia para o manifesto.
-

Manifesto

🌀 Problema: como modelar um resumo:

- Composto por uma lista ordenada de Parágrafos;
- Onde a ordem importa semanticamente.

M1. Ordem explícita: `rdf:List`

Podemos usar o padrão canónico de listas ordenadas em RDF:

```
@prefix : <http://example.org/doc#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

:Resumo1 a :Resumo ;
  :temParagrafos (
    :Paragrafo1
    :Paragrafo2
    :Paragrafo3
  ) .

:Paragrafo1 a :Paragrafo ;
  :texto "Introdução ao tema." .

:Paragrafo2 a :Paragrafo ;
  :texto "Desenvolvimento da ideia principal." .

:Paragrafo3 a :Paragrafo ;
  :texto "Conclusão do resumo." .
```

✅ Vantagens

- Ordem linear sem ambiguidades;
- Compatível com RDF, OWL, SPARQL;
- Muito bem suportado por ferramentas.

⚠ Desvantagens

- Verboso internamente (`rdf:first`, `rdf:rest`).

👉 Esta é a melhor opção quando a ordem é essencial, como num resumo textual.

M2. Parágrafos com índice explícito: `xsd:integer`

Modelo aparentemente mais simples e muito usado:

```
:Resumo1 a :Resumo ;
      :temParagrafo :P1, :P2, :P3 .

:P1 a :Paragrafo ;
  :ordem 1 ;
  :texto "Introdução ao tema." .

:P2 a :Paragrafo ;
  :ordem 2 ;
  :texto "Desenvolvimento da ideia principal." .

:P3 a :Paragrafo ;
  :ordem 3 ;
  :texto "Conclusão do resumo." .
```

✅ Vantagens

- Simples (não envolve processamento de estruturas de dados);
- Fácil de consultar com SPARQL (`ORDER BY`);
- Fácil de manter.

⚠ Desvantagens

- A ordem não é estrutural, é convencional (é mantida pela semântica de um dos campos);
- OWL não consegue garantir unicidade/continuidade dos índices.

👉 Boa opção se vais gerar e consumir via aplicação.

M3. Relações encadeadas (ligação ao próximo)

```
:P1 :seguidoPor :P2 .
:P2 :seguidoPor :P3 .
```

❌ Não recomendada para este caso

- Difícil de consultar;
- Não garante início/fim;
- Propensa a ciclos;
- Fraca interoperabilidade.

Só faz sentido para streams ou versões temporais.

Exemplo: Reconstrução do resumo em SPARQL usando a primeira abordagem

```
SELECT ?texto
WHERE {
  :Resumo1 :temParagrafos/rdf:rest*/rdf:first ?p .
  ?p :texto ?texto .
}
```

Passando as restrições para SHACL

🎯 Objetivos de validação

- Queremos garantir que:

1. Um Resumo:

- Tem exatamente uma lista de parágrafos;
- A lista não está vazia.

2. Cada item da lista:

- É um Parágrafo.

3. Cada Parágrafo:

- Tem exatamente um texto;
- O texto é uma string não vazia.

4. A estrutura `rdf:List` é bem formada.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix : <http://example.org/doc#> .

#####
# Shape para Resumo
#####

:ResumoShape
  a sh:NodeShape ;
  sh:targetClass :Resumo ;

  sh:property [
    sh:path :temParagrafos ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:node :RdfListShape ;
    sh:message "Um Resumo deve ter exatamente uma lista ordenada de
parágrafos."
  ] .

#####
```

```

# Shape genérico para rdf:List
#####

:RdfListShape
  a sh:NodeShape ;

  sh:or (
    [ sh:hasValue rdf:nil ]
    [
      sh:and (
        [
          sh:property [
            sh:path rdf:first ;
            sh:minCount 1 ;
            sh:maxCount 1 ;
            sh:node :ParagrafoShape ;
            sh:message "Cada elemento da lista deve ser um
Parágrafo."
          ]
        ]
        [
          sh:property [
            sh:path rdf:rest ;
            sh:minCount 1 ;
            sh:maxCount 1 ;
            sh:node :RdfListShape ;
            sh:message "rdf:rest deve apontar para outra lista
RDF válida."
          ]
        ]
      )
    ]
  ) .

#####
# Shape para Parágrafo
#####

:ParagrafoShape
  a sh:NodeShape ;
  sh:targetClass :Paragrafo ;

  sh:property [
    sh:path :texto ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:minLength 1 ;
    sh:message "Um Parágrafo deve ter exatamente um texto não vazio."
  ] .

```

Geração de uma ontologia a partir de um dataset em JSON

Analisa o seguinte dataset:

```
[
  {
    "designacao": "Lista de compras 1",
    "data": "2021-01-18",
    "produtos": [
      {
        "designacao": "Postas de Salmão",
        "categoria": "Peixe",
        "quantidade": {
          "valor": "4",
          "unidade": "unidade"
        }
      },
      {
        "designacao": "Embalagem de SKIP",
        "categoria": "Limpeza",
        "quantidade": {
          "valor": "3",
          "unidade": "unidade"
        }
      },
      ...
    ]
  },
  ...
]
```

Como podes ter observado, o dataset consiste numa lista de listas de compras.

Realiza agora as seguintes tarefas:

1. Especifica o modelo ontológico para suportar esta informação: classes, atributos e relações;
2. Cria uma script em Python para gerar os indivíduos automaticamente a partir do dataset;
3. Junta tudo no Protégé e analisa o resultado.

TPC2: Criar uma ontologia para EMD e povoá-la

- Analisa o dataset dos EMD;
- Cria com o protégé a parte estrutural da ontologia;
- Cria uma script em Python para a povoar.